Lab 5–6 — Hồi quy tuyến tính và Gradient Descent

1. Đề bài tiếng Anh

LAB SESSION 5 & 6

Problem Description

In this project, you will build a simple machine learning model to predict students' exam scores based on their study hours.

You will use Python, NumPy, Matplotlib, and SymPy to explore mathematical concepts used in machine learning, including **partial derivatives, gradients, gradient descent, and the Hessian matrix**.

Dataset

Study Hours (X)	Exam Score (y)
1	35
2	40
3	50
4	55
5	65
6	70
7	80
8	85

Requirements

Task 1: Create and Visualize the Dataset

- Create two NumPy arrays: `X` (students' study hours) and `y` (students' exam scores).
- Use Matplotlib to create a scatter plot of study hours versus exam scores.
- Add an appropriate title, axis labels, and grid.

Task 2: Build a Simple Linear Prediction Model

Use the following model:

$$ \hat{y} = wX+b $$

where `w` is the weight, `b` is the bias, and ŷ is the predicted exam score.

- Implement a `predict(X, w, b)` function.
- Implement a `compute_loss(y_true, y_pred)` function using Mean Squared Error (MSE):

$$ \mathrm{MSE}=\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat y_i)^2. $$

- Initialize `w = 0` and `b = 0`. Calculate/display initial predictions and MSE.

Task 3: Calculate Partial Derivatives and the Hessian

Consider the loss function for one sample:

$$ L(w,b)=(2w+b-5)^2. $$

- Calculate the partial derivative of `L` with respect to `w`.
- Calculate the partial derivative of `L` with respect to `b`.
- Construct the gradient vector.
- Calculate/display the Hessian matrix using SymPy.

Task 4: Implement Gradient Descent

Implement Gradient Descent from scratch to train the model on the dataset above.

For MSE, use:

$$ \frac{\partial L}{\partial w}=\frac{2}{n}\sum_i X_i(wX_i+b-y_i),\qquad \frac{\partial L}{\partial b}=\frac{2}{n}\sum_i(wX_i+b-y_i). $$

Update parameters by subtracting the learning rate multiplied by each gradient component.

- Initialize `w = 0`, `b = 0`.
- Set the learning rate to `0.01` and the number of epochs to `1000`.
- Calculate gradients and update `w`, `b` each epoch.
- Record MSE during training.
- Display final `w`, `b`, and MSE.

Task 5: Analyze the Gradient and Hessian of the Full Dataset Loss

Apply the full-dataset loss:

$$ L(w,b)=\frac{1}{n}\sum_i(y_i-wX_i-b)^2. $$

- Calculate/display the gradient vector and Hessian matrix.
- Evaluate the gradient at the trained `w` and `b`.
- Calculate Hessian eigenvalues using NumPy.
- Briefly explain what gradient and Hessian tell us about optimization.

Chuyển từ `LAB SESSION 5_6.docx`. Các biểu thức toán vốn là đối tượng Word/hình công thức được trình bày lại bằng LaTeX để Markdown hiển thị ổn định. Dataset gốc có 8 hàng, được giữ nguyên theo đề.

2. Đề bài tiếng Việt

LAB SESSION 5 & 6 — Hồi quy tuyến tính và Gradient (độ dốc của hàm) Descent (thuật toán giảm độ dốc)

Mô tả bài toán

Xây dựng một mô hình học máy đơn giản để dự đoán **điểm thi của sinh viên** dựa vào **số giờ học**. Dùng Python, NumPy (thư viện tính toán mảng), Matplotlib (thư viện vẽ đồ thị) và SymPy (thư viện toán ký hiệu) để tìm hiểu đạo hàm riêng, gradient, thuật toán Gradient Descent và ma trận Hessian (ma trận đạo hàm bậc hai).

Dữ liệu

Số giờ học (X, giờ)	Điểm thi (y, điểm)
1	35
2	40
3	50
4	55
5	65
6	70
7	80
8	85

Yêu cầu

Bài 1 — Tạo và trực quan hóa dữ liệu Tạo hai mảng NumPy (thư viện tính toán mảng) `X` và `y` chứa số giờ học và điểm. Dùng Matplotlib (thư viện vẽ đồ thị) vẽ scatter plot (biểu đồ phân tán) giờ học so với điểm thi; đặt tiêu đề, nhãn hai trục và lưới.

Bài 2 — Xây dựng mô hình dự đoán tuyến tính Sử dụng mô hình **ŷ = wX + b**, trong đó `w` là trọng số, `b` là hệ số chệch, ŷ là điểm dự đoán. Viết `predict(X,w,b)` và `compute_loss(y_true,y_pred)` dùng MSE (trung bình bình phương sai số):

[\mathrm{MSE}=\frac{1}{n}\sum_{i=1}^{n}(y_i-\hat y_i)^2.]

Khởi tạo `w=0`, `b=0`, hiển thị dự đoán đầu tiên và MSE (trung bình bình phương sai số) ban đầu.

Bài 3 — Đạo hàm riêng và Hessian (ma trận đạo hàm bậc hai) Với hàm một mẫu **L(w,b)=(2w+b−5)²**, tính đạo hàm riêng theo `w`, theo `b`, ghép vector gradient (độ dốc của hàm) và tính ma trận Hessian bằng SymPy (thư viện toán ký hiệu).

Bài 4 — Cài đặt Gradient (độ dốc của hàm) Descent (thuật toán giảm độ dốc) Tự huấn luyện mô hình theo MSE (trung bình bình phương sai số) trên dữ liệu đã cho. Dùng **learning rate (tốc độ học) = 0,01**, **epochs (lượt học) = 1000**, khởi tạo `w=b=0`. Mỗi epoch tính gradient:

[\frac{\partial L}{\partial w}=\frac2n\sum_i X_i(wX_i+b-y_i),\qquad \frac{\partial L}{\partial b}=\frac2n\sum_i(wX_i+b-y_i).]

Sau đó trừ learning rate nhân gradient để cập nhật hai tham số, ghi lại MSE qua mỗi epoch. Hiển thị `w`, `b`, MSE cuối cùng.

Bài 5 — Gradient (độ dốc của hàm)/Hessian (ma trận đạo hàm bậc hai) cho toàn bộ dữ liệu Với **L(w,b) = (1/n) Σᵢ (yᵢ−wXᵢ−b)²**, tính và hiển thị gradient vector, Hessian. Thay hai giá trị `w,b` đã học vào gradient và tính các eigenvalue (trị riêng) của Hessian bằng NumPy (thư viện tính toán mảng). Giải thích gradient và Hessian cho biết điều gì về quá trình tối ưu.

3. Code Python có chú thích

```
import sys
from pathlib import Path
import numpy as np
import sympy as sp

X = np.array([1,2,3,4,5,6,7,8], dtype=float)
Y = np.array([35,40,50,55,65,70,80,85], dtype=float)

def predict(x,w,b):
    return w*x+b

def compute_loss(y_true,y_pred):  # MSE là trung bình bình phương sai số dự đoán.
    return float(np.mean((y_true-y_pred)**2))

def task01_dataset():
    if not (X.shape == Y.shape == (8,)): raise ValueError("Kiểm tra dữ liệu không đạt")
    return {"hours":X.tolist(),"scores":Y.tolist()}

def task02_initial_model():
    guesses = predict(X,0.0,0.0)
    initial_loss = compute_loss(Y,guesses)
    return guesses,initial_loss

def task03_symbolic_single():  # Dùng SymPy để tìm đạo hàm, không đoán gradient.
    w,b=sp.symbols("w b",real=True)
    loss=(2*w+b-5)**2
    variables=(w,b)
    gradient=sp.Matrix([sp.diff(loss,v) for v in variables])
    hessian=sp.hessian(loss,variables)
    return loss,gradient,hessian

def task04_train(learning_rate=0.01,epochs=1000):  # Lặp cập nhật w và b theo chiều ngược gradient.
    w=b=0.0
    history=[]
    n=X.size
    for _ in range(epochs):
        residual=predict(X,w,b)-Y
        dw=(2/n)*np.dot(X,residual)
        db=(2/n)*residual.sum()  # Đếm hoặc cộng phần tử để tính tổng/tỷ lệ.
        w-=learning_rate*dw
        b-=learning_rate*db
        history.append(compute_loss(Y,predict(X,w,b)))  # Thêm phần tử vào cuối danh sách bằng append (thêm cuối).
    return w,b,np.array(history)

def task05_symbolic_full(trained_w,trained_b):  # So sánh gradient và Hessian khi mô hình có hai tham số.
    w,b=sp.symbols("w b",real=True)
    values=[sp.Integer(int(x)) for x in X]
    targets=[sp.Integer(int(y)) for y in Y]
    n=len(values)
    full_loss=sum((yi-w*xi-b)**2 for xi,yi in zip(values,targets))/n
    gradient=sp.Matrix([sp.diff(full_loss,w),sp.diff(full_loss,b)])
    hessian=sp.hessian(full_loss,(w,b))
    gradient_at=gradient.subs({w:trained_w,b:trained_b}).evalf()
```

```
    eigvals=np.linalg.eigvalsh(np.asarray(hessian,dtype=float))
    return gradient,hessian,gradient_at,eigvals

def save_plots(w,b,losses):
    import matplotlib
    matplotlib.use("Agg")
    import matplotlib.pyplot as plt
    root=Path(__file__).resolve().parent/"outputs"
    root.mkdir(exist_ok=True)
    fig,ax=plt.subplots()
    ax.scatter(X,Y,label="Observed scores")
    ax.plot(X,predict(X,w,b),label="Fitted prediction")
    ax.set(xlabel="Study hours",ylabel="Exam score (points)",title="Hours vs. exam scores")
    ax.grid(True)
    ax.legend()
    fig.savefig(root/"hours_vs_scores.png",dpi=140,bbox_inches="tight")
    plt.close(fig)  # Đóng biểu đồ sau khi dùng để giải phóng bộ nhớ.
    fig,ax=plt.subplots()
    ax.plot(np.arange(1,len(losses)+1),losses)
    ax.set(xlabel="Epoch",ylabel="MSE (squared score units)",title="Gradient Descent Training")
    ax.grid(True)
    fig.savefig(root/"loss_curve.png",dpi=140,bbox_inches="tight")
    plt.close(fig)  # Đóng biểu đồ sau khi dùng để giải phóng bộ nhớ.
    print("Wrote",root/"hours_vs_scores.png","and",root/"loss_curve.png")

def main():
    task01_dataset()
    initial_preds,initial_mse=task02_initial_model()
    example_loss,example_grad,example_hessian=task03_symbolic_single()
    w,b,history=task04_train(learning_rate=0.01,epochs=1000)
    full_grad,full_hessian,at_final,eigenvalues=task05_symbolic_full(w,b)
    final_mse=compute_loss(Y,predict(X,w,b))
    if not (history.shape==(1000,)): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (final_mse < initial_mse): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (all(eigenvalues > 0)): raise ValueError("Kiểm tra dữ liệu không đạt")
    if not (np.isclose(final_mse,history[-1])): raise ValueError("Kiểm tra dữ liệu không đạt")
    print("Initial predictions:",initial_preds)
    print("Initial MSE:",round(initial_mse,3))
    print("Single-sample symbolic loss:",example_loss)
    print("Single-sample gradient:",example_grad,"Hessian:",example_hessian)
    print(f"Trained w={w:.6f}, b={b:.6f}, final MSE={final_mse:.6f}")
    print("Full gradient:",full_grad)
    print("Full Hessian:",full_hessian)
    print("Gradient evaluated at trained parameters:",at_final)
    print("Hessian eigenvalues:",eigenvalues)
    if "--no-plot" not in sys.argv:
        save_plots(w,b,history)
if __name__=="__main__":
    main()
```

4. Giải thích nội dung và cách giải

Đây là phần diễn giải cho **một chương trình hoàn chỉnh ở Mục 3**. Các ví dụ code ngắn lặp lại trước đây được chuyển thành giải thích bằng lời để khi in không phải theo dõi nhiều phiên bản code.

Câu 1

Mô hình dữ liệu: `X=[1,2,3,4,5,6,7,8]` giờ; `y=[35,40,50,55,65,70,80,85]` điểm. Biểu đồ scatter vẽ `X` theo trục ngang và `y` theo trục đứng.

Vì sao cần nhìn dữ liệu trước? Nếu giờ học và điểm thi không có xu hướng liên quan, mô hình đường thẳng có thể không phù hợp. Vẽ các điểm trước giúp người học hiểu dữ liệu, phát hiện giá trị bất thường và hiểu trục nào là đầu vào.

Quy trình: (1) Tạo hai mảng `X` (giờ) và `Y` (điểm). (2) Kiểm tra chúng dài bằng nhau, vì mỗi giờ phải đi cùng một điểm. (3) Vẽ scatter (biểu đồ phân tán), gắn nhãn và lưới. **Cú pháp và tham số:** `dtype=float` chọn số thực; `plt.subplots()` trả một figure (khung hình) và axes (vùng vẽ); `ax.scatter(x, y)` vẽ cặp điểm; `xlabel`, `ylabel` là nhãn trục, `title` là tiêu đề; `grid(True)` bật lưới. Để chạy riêng đoạn minh họa cần nạp `import matplotlib.pyplot as plt`; code cuối bài chỉ vẽ khi không truyền `--no-plot`.

Kiến thức/công thức trọng tâm: giữ nguyên 8 dòng dữ liệu X=1..8 và điểm tương ứng; vẽ scatter và đường hồi quy theo giờ/điểm.

Ví dụ và lỗi thường gặp: Giữ nguyên tám cặp giờ học và điểm từ đề. Trên biểu đồ, trục ngang đo giờ học, trục dọc đo điểm.

Thuật ngữ cần nhớ: Dataset (bộ dữ liệu).

Câu 2

`predict()` nhân từng số giờ với `w`, cộng `b`; `compute_loss()` tính từng hiệu điểm thật-dự đoán, bình phương, lấy trung bình. Khởi tạo cả hai bằng 0 để có đường chuẩn trước khi học.

Vì sao dùng MSE? Mô hình dự đoán bằng đường thẳng cần một cách đo mức sai khác so với điểm thật. Bình phương sai số làm số âm và số dương đều tăng loss; lấy trung bình giúp so sánh qua nhiều dữ liệu.

Quy trình: (1) Đặt tham số hệ số `w` và độ dịch `b`. (2) Tính điểm dự đoán theo giờ học. (3) Lấy điểm thật trừ điểm dự đoán. (4) Bình phương từng sai số rồi lấy trung bình. (5) Đối chiếu mô hình ban đầu `w=0`, `b=0`. **Cú pháp:** `w * X` nhân mọi phần tử của mảng giờ với cùng hệ số, `b` cộng độ dịch vào từng kết quả; `** 2` là bình phương (không dùng `^`); `np.mean` lấy trung bình, cho MSE có đơn vị điểm bình phương. `w=0` và `b=0` chỉ là cách khởi tạo, chưa phải giá trị tốt nhất.

Kiến thức/công thức trọng tâm: MSE = $(1/n)\sum(y-\hat{y})^2$; $\hat{y}=wX+b$; đơn vị MSE là điểm^2.

Ví dụ và lỗi thường gặp: Điểm dự đoán là $\hat{y}=wX+b$; MSE (trung bình bình phương sai số) lấy trung bình của bình phương sai khác. Khi w=b=0, điểm dự đoán đều bằng 0, tạo mốc trước huấn luyện.

Thuật ngữ cần nhớ: Prediction (dự đoán).

Câu 3

Dùng `sp.symbols` tạo biến biểu thức, `sp.diff` lấy đạo hàm, `sp.hessian` trả ma trận đạo hàm bậc hai. Kiểm chứng: gradient (độ dốc của hàm) của biểu thức mẫu có thành phần theo w và b lần lượt bằng `4(2w+b−5)` và `2(2w+b−5)`.

Vì sao tính đạo hàm riêng và Hessian? Hàm loss của một điểm phụ thuộc cả `w` và `b`. Gradient cho biết loss thay đổi theo từng tham số; Hessian cho biết độ cong, giúp giải thích vì sao có hướng thay đổi nhanh/chậm.

Quy trình: (1) Khai báo hai biến ký hiệu. (2) Xây biểu thức loss với đầu vào 2 và điểm thật 5. (3) Lấy đạo hàm theo mỗi biến. (4) Gom đạo hàm thành vector và tính ma trận đạo hàm bậc hai. **Hàm và flag:** `sp.symbols(..., real=True)` khai báo biến thực; `sp.diff(loss, w)` lấy đạo hàm riêng theo `w`; `sp.Matrix` tạo vector cột; `sp.hessian(loss, (w,b))` trả ma trận theo đúng thứ tự biến. `real=True` là giả định số thực trong SymPy. Ở một điểm dữ liệu này, Hessian có một hướng độ cong bằng 0; không kết luận loss có cực tiểu duy nhất từ một điểm.

Kiến thức/công thức trọng tâm: đạo hàm riêng SymPy (thư viện toán ký hiệu) `diff`; Hessian (ma trận đạo hàm bậc hai) ví dụ một điểm $L=(2w+b-5)^2$ có gradient $[4(2w+b-5),2(2w+b-5)]$, Hessian $[[8,4],[4,2]]$.

Ví dụ và lỗi thường gặp: Gradient (độ dốc của hàm) gồm đạo hàm riêng bậc nhất theo w và b; Hessian (ma trận đạo hàm bậc hai) là ma trận đạo hàm bậc hai. Phân biệt loss (hàm mất mát) một điểm ở Bài 3 với loss toàn bộ tập dữ liệu.

Thuật ngữ cần nhớ: Partial derivatives (đạo hàm riêng).

Câu 4

Mỗi vòng dự đoán điểm, lấy phần dư `predict(X,w,b)-Y`, tính gradient (độ dốc của hàm) `dw,db`, trừ learning_rate nhân gradient, lưu MSE (trung bình bình phương sai số) mới; không tạo thuật toán khác với script tham khảo.

Vì sao cập nhật nhiều bước? Dự đoán ban đầu chưa khớp dữ liệu. Gradient cho biết cần điều chỉnh `w` và `b` về hướng nào để giảm MSE, còn tốc độ học quyết định mỗi lần điều chỉnh bao nhiêu.

Quy trình: (1) Khởi tạo `w=b=0`. (2) Tính sai số dự đoán với toàn bộ giờ học. (3) Tính gradient đối với `w` bằng tích vô hướng với `X`, gradient của `b` bằng tổng sai số. (4) Trừ tốc độ học nhân gradient khỏi mỗi tham số. (5) Lặp đúng số vòng, ghi lịch sử MSE để so sánh. **Giải thích tham số/toán tử:** `learning_rate=0.01` là kích cỡ bước; `epochs=1000` là số lần lặp; `range(epochs)` tạo đúng 1.000 lượt; `_` là biến lặp không dùng; `np.dot` nhân và cộng các cặp phần tử; `X.size` là số quan sát; `-=` viết ngắn cho phép trừ rồi cập nhật. Tốc độ học quá lớn có thể làm loss tăng; kiểm tra lịch sử loss để phát hiện.

Kiến thức/công thức trọng tâm: mỗi epoch cập nhật w,b trừ 0.01×gradient; tính lại MSE, lưu đủ 1000 epoch; gradient $w=(2/n)\ \Sigma\ X(wX+b-y)$, $b=(2/n)\ \Sigma(wX+b-y)$.

Ví dụ và lỗi thường gặp: Mỗi epoch (lượt cập nhật) tính phần dư, hai gradient (độ dốc của hàm) dw, db, rồi trừ learning rate (tốc độ học) nhân gradient. Giữ nguyên 1000 epoch và tốc độ học 0,01.

Thuật ngữ cần nhớ: Gradient Descent (thuật toán giảm độ dốc).

Câu 5

Biểu thức loss (hàm mất mát) tổng được tạo từ 8 giá trị gốc trong `X,Y`; SymPy (thư viện toán ký hiệu) tự tính gradient (độ dốc của hàm) và Hessian (ma trận đạo hàm bậc hai). `subs` thay tham số học được, `np.linalg.eigvalsh` cho trị riêng ma trận đối xứng. Không lẫn với Hessian một điểm ở bài 3.

Vì sao tính trên toàn bộ dữ liệu? Đạo hàm từ một điểm ở câu trước không phản ánh đủ tám cặp giờ/điểm. Ở đây phải lập loss trung bình của **tất cả tám điểm**, rồi xem gradient còn lớn hay nhỏ tại tham số đã học và Hessian mô tả độ cong.

Quy trình: (1) Tạo loss là trung bình bình phương sai số của tám điểm. (2) Lấy đạo hàm theo `w` và `b` bằng SymPy. (3) Thay tham số cuối vào gradient để kiểm tra mức gần tối ưu. (4) Tính Hessian và trị riêng để thấy độ cong theo các hướng khác nhau. **Giải thích tham số và cú pháp:** `zip(X,Y)` ghép từng giờ học với đúng điểm; `sum` cộng tám bình phương; `len(X)` chia thành trung bình; `.subs({w: ..., b: ...})` thay tham số vào biểu thức; `.evalf()` tính số gần đúng. `np.asarray(..., dtype=float)` đổi SymPy sang mảng số; `np.linalg.eigvalsh` tính trị riêng của **ma trận đối xứng** như Hessian trong bài. `trained_w` và `trained_b` là kết quả sau bước huấn luyện ở Câu 4.

Kiến thức/công thức trọng tâm: hàm loss đầy đủ 8 mẫu khác Hessian một mẫu; tính eigenvalue (trị riêng) bằng `eigvalsh` vì Hessian đối xứng; so gradient ở nghiệm đã học.

Ví dụ và lỗi thường gặp: Gradient (độ dốc của hàm) cho biết hướng tăng loss (hàm mất mát); Hessian (ma trận đạo hàm bậc hai) và các trị riêng cho biết độ cong. Đánh giá tại tham số đã học chứ không dùng w,b ngẫu nhiên.

Thuật ngữ cần nhớ: Hessian (ma trận đạo hàm bậc hai).

Hiểu một lượt huấn luyện Gradient Descent

Đầu vào là các cặp `X` và `Y`, trong đó `X` là biến mô tả và `Y` là giá trị cần dự đoán. Mô hình tuyến tính dự đoán `ŷ = w × x + b`; `w` điều khiển độ dốc, `b` là điểm cắt trục tung. Hàm `compute_loss` đo **MSE** bằng trung bình bình phương phần chênh giữa dự đoán và giá trị thật; MSE càng nhỏ thì mô hình càng khớp *trên bộ dữ liệu đang xét*, chưa đủ bảo đảm khái quát tốt.

Chu trình cập nhật gồm: tính dự đoán từ `w,b` hiện tại; tính sai số/loss; tìm đạo hàm riêng `dw, db`; sau đó cập nhật `w -= learning_rate × dw`, `b -= learning_rate × db`. Đây là một bước; `epochs` là số vòng lặp huấn luyện, còn `learning_rate` quyết định độ dài mỗi bước. Tốc độ học quá lớn có thể vượt qua điểm cực tiểu; quá nhỏ có thể cần rất nhiều vòng.

Vai trò của SymPy: `sp.symbols` khai báo biến ký hiệu, `sp.diff` lấy đạo hàm, `sp.Matrix` gom gradient thành vector, còn `hessian` mô tả độ cong thông qua đạo hàm bậc hai. Đạo hàm được tính tượng trưng là cơ sở để đối chiếu hướng và kích thước bước cập nhật trong code NumPy.

Khi đọc đồ thị: trục ngang là số vòng huấn luyện, trục dọc là MSE; đường giảm thường cho thấy thuật toán đang cải thiện trên dữ liệu luyện. Nếu loss tăng hoặc dao động, thử xem lại `learning_rate`, dữ liệu đầu vào và việc tính gradient trước khi tăng `epochs`.